PROJECTE 1 – FASE II
Mòdul: M0616- Projecte de desenvolupament web **Professor:** Albert Guardiola **Data límit:** **ENTREGA4 – 21/10/26** **ENTREGA5 – 28/10/26** **Mètode d'entrega:** Per mitjà del Clickedu de l'assignatura.

PROJECTE 1 – FASE II

Desenvolupament Assistit amb GitHub Copilot i IA

Context i Objectius

En el desenvolupament web modern, les eines d'Intel·ligència Artificial assistida com **GitHub Copilot** s'han integrat directament en el cicle de vida del programari (DevOps). No es tracta només de generar codi automàticament, sinó d'establir regles de desenvolupament a nivell de repositori, automatitzar la documentació de les Pull Requests i millorar la revisió de codi.

En aquesta ampliació de la pràctica, aprendràs a configurar el teu repositori DevTasks perquè la IA treballi de manera alineada amb l'arquitectura del projecte (separació de lògica de negoci i DOM) i aplicaràs aquests coneixements en el teu flux de treball diari.

ENTREGA 4. GitHub Skills (IA i Copilot)

Abans d'aplicar la configuració al teu repositori, hauràs de realitzar els tutorials oficials de la plataforma **GitHub Skills** (disponibles 100% de franc amb el compte normal de GitHub Free):

GitHub Skill	Coneixement adquirit	Aplicació al projecte DevTasks
Getting Started with GitHub Copilot	Compleció de codi, generació de funcions i ús del xat assistent a l'IDE.	Generació de noves funcions de negoci i la seva suite de tests unitaris.
Customize Your GitHub Copilot Experience	Creació i configuració d'instruccions personalitzades del repositori.	Creació del fitxer .github/copilot-instructions.md amb les regles d'arquitectura.
Copilot Code Review	Revisió assistida de codi i automatització de resums de canvis.	Generació de resums automàtics i revisions a les Pull Requests.

Entrega un document PDF que inclogui per a cada un dels 3 tutorials:

- **Captures de pantalla d'evidència:** On es demostri clarament que l'activitat s'ha completat satisfactòriament.
- **Reflexió individual:** Què has aprendre, quin problema resol en l'enginyeria de programari i com s'aplica específicament al repositori DevTasks.

ENTREGA 5 — Evidència d'Integració al Repositori Propi

Entrega de la implementació pràctica al repositori GitHub del projecte. Caldrà lliurar un document PDF amb les evidències de la integració i el flux complet de treball.

Guia Pas a Pas Detallada per a la Integració Local i Remota

PAS 1. Preparar l'Entorn de Treball (VS Code)

1. Obre el teu projecte DevTasks a Visual Studio Code.
2. Si la finestra/icona de GitHub Copilot ja apareix integrada per defecte a la interfície de VS Code, no cal instal·lar cap extensió addicional. En cas contrari, obre la pestanya d'extensions (Ctrl+Shift+X) i instal·la **GitHub Copilot** i **GitHub Copilot Chat**.
3. Fes clic a la icona del teu compte (esquina inferior esquerra) i selecciona **"Sign in to use GitHub Copilot"** per vincular el teu compte de GitHub.

PAS 2. Crear les Instruccions Personalitzades del Repositori (.github/copilot-instructions.md)

1. Dins de la carpeta .github/ del teu projecte, crea un fitxer anomenat copilot-instructions.md.
2. Aquest fitxer estableix les directrius de codi del repositori. Copilot el llegirà automàticament com a context primari per no generar mai codi que trenqui l'arquitectura.
3. Copia i enganxa el contingut següent:

```
# Instruccions de Desenvolupament per a DevTasks

1. Arquitectura de Mòduls i Desacoblament:
   - Tota la lògica de negoci (càlculs, validacions, filtratges) ha d'anar ÚNICAMENT a `js/taskManager.js` utilitzant ES6 Modules (`export`).
   - Les funcions de `taskManager.js` han de ser pures (sense referències a `document`, `window` o `localStorage`).
   - La manipulació del DOM, esdeveniments d'interfície i persistència corresponen exclusivament a `js/app.js`.

2. Tests Unitaris Amb Vitest:
   - Per a qualsevol nova funció o modificació realitzada a `taskManager.js`, crea obligatòriament les seves proves unitàries a `tests/app.test.js` fent servir Vitest (`describe`, `it`, `expect`).

3. Estil i Qualitat de Codi:
   - Utilitza JavaScript modern (ES6+), funcions clares, noms de variables descriptius i evita l'ús de variables globals no desitjades.
```

PAS 3. Crear el Template per a les Pull Requests (.github/PULL_REQUEST_TEMPLATE.md)

1. Crea el fitxer PULL_REQUEST_TEMPLATE.md dins de la carpeta .github/.
2. Aquest fitxer definirà l'estructura predeterminada que s'obrirà cada vegada que un desenvolupador cree una nova Pull Request al repositori.
3. Copia i enganxa la següent estructura Markdown:

```
## 📝 Descripció dels Canvis
<!-- Explica breument el propòsit d'aquesta PR -->

## 🤖 Resum Assistit per IA / Copilot
<!-- Genera o pega aquí el resum produït per Copilot -->

## 🔨 Verificació de Tests i Qualitat
- [ ] Els tests unitaris s'executen localment amb `npm test` i passen correctament.
```

```
- [ ] El codi generat respecta les regles de `.github/copilot-instructions.md`.
```

PAS 4. Crear una Nova Funcionalitat Assistida per Copilot

1. Obre el terminal integrat de VS Code i crea una nova branca de funcionalitat:

   git checkout -b feature/copilot-clear-completed

2. Obre el xat de **GitHub Copilot Chat** a la barra lateral de VS Code (o prement Ctrl+Alt+I).
3. Escriu el següent prompt per demanar la nova funció de negoci:

"Tenint en compte el fitxer .github/copilot-instructions.md, crea una nova funció pura exportada a js/taskManager.js anomenada clearCompletedTasks(tasks) que rebi un array de tasques i retorni un nou array filtrat sense les tasques completades."

4. Revisa el codi suggerit per Copilot, accepta'l i desa el fitxer js/taskManager.js.
5. A continuació, demana a Copilot Chat que et generi el test unitari corresponent:

"Genera el test unitari amb Vitest per a la funció clearCompletedTasks(tasks) i afegeix-lo a tests/app.test.js."

6. Executa la suite de proves al terminal per confirmar que tot funciona:

   npm test

7. Fes commit dels canvis i puja la nova branca al repositori remot:

   git add .

   git commit -m "feat: afegida funció clearCompletedTasks i tests unitaris amb Copilot"

   git push -u origin feature/copilot-clear-completed

PAS 5. Crear la Pull Request i Documentar amb IA

1. Entra al teu repositori a GitHub.com i crea una nova Pull Request des de la branca feature/copilot-clear-completed cap a main.
2. Veuràs que el formulari s'ha emplenat automàticament amb l'estructura de PULL_REQUEST_TEMPLATE.md.
3. Utilitza Copilot per emplenar la secció de resum de la PR (segons si utilitzes el pla Student o el pla Free, com s'explica més endavant).
4. **Execució de Copilot Code Review al Núvol (Pla Student / Pro):** Al menú lateral dret de la PR a GitHub.com, ves a la secció *Reviewers* i afegeix @copilot com a revisor. Observa com el model de IA analitza el diff de la PR al núvol i publica comentaris directament a les línies de codi. *(Si utilitzes el pla Free, aplica l'alternativa de Code Review des de VS Code explicada més endavant)*.
5. Comprova que el workflow de CI (GitHub Actions) s'inicia automàticament i passa els tests satisfactòriament (check verd).

Adaptacions i Alternatives Detallades per a Estudiants amb Pla Free

Si no disposeu de la llicència **GitHub Student Developer Pack** i esteu utilitzant el **pla GitHub Free convencional**, podeu dur a terme la pràctica exactament amb el mateix nivell de rigor adaptant els passos segons la següent taula d'equivalències:

Funcionalitat	**Opció amb Pla Student / Pro**	**Alternativa Pas a Pas en Pla Free (Sense Pagar)**
1. Resum de la PR (*PR Summarization*)	Botó automàtic **"Summary with Copilot"** directament a la interfície web de GitHub.com.	**Procediment alternatiu:** 1. A VS Code, selecciona la pestanya de Source Control o el teu git diff. 2. Obre Copilot Chat i escriu: *"Genera un resum executiu en format Markdown de tots els canvis que he realitzat en aquesta branca per incloure'l a la meva Pull Request."* 3. Copia el text generat i enganxa'l manualment a la secció corresponent del template de la PR a GitHub.com.
2. Revisió de Codi (*Code Review*)	Funció **"Request Copilot Review"** integrada a la pestanya de fitxers modificats de la PR web (o afegint @copilot com a reviewer).	**Procediment alternatiu:** 1. Obre el fitxer modificat (js/taskManager.js) a VS Code. 2. Obre Copilot Chat i escriu: *"Fes una revisió de codi d'aquesta funció segons les regles de .github/copilot-instructions.md. Indica si compleix l'arquitectura i suggereix possibles*

Funcionalitat	Opció amb Pla Student / Pro	Alternativa Pas a Pas en Pla Free (Sense Pagar)
		millores." 3. Adjunta una captura d'aquesta anàlisi al document PDF d'evidències de l'ENTREGA 5.
3. Gestió de Quota Mensual	Peticions il·limitades de completat de codi i xat.	**Optimització de consum:** El pla Free inclou un cupó mensual limitat de missatges de xat i suggeriments. Utilitza prompts directes i concrets (evitant consultes vagues) per estalviar interaccions i no esgotar la quota durant la pràctica.

Evidències Requerides per a l'ENTREGA 5 (Document PDF)

El document PDF d'evidències de la integració haurà d'incloure obligatòriament les següents captures estructurades:

1. **Configuració del Repositori:** Captura dels fitxers .github/copilot-instructions.md i .github/PULL_REQUEST_TEMPLATE.md creats al repositori.
2. **Generació de Codi i Test:** Captura de pantalla de VS Code on es vegi l'ús de Copilot Chat generant la funció i la suite de proves a tests/app.test.js.
3. **Execució Local:** Captura del terminal executant la comanda npm test amb tots els tests superats en verd.
4. **Pull Request i Code Review amb IA:** Captura de la PR real a GitHub.com on es vegi:
    - El resum de la IA emplenat al template.
    - El Code Review executat per la IA (ja sigui la revisió de @copilot a la web de GitHub o la captura de l'anàlisi de VS Code si s'utilitza el pla Free).
    - El check de CI (GitHub Actions) superat en verd.
5. **Enllaç al Repositori:** L'enllaç públic o accessible al teu repositori de GitHub.